Database Concurrency - Exercises

Setup

Before starting the exercises, create the test environment:

```
-- Create a test database for concurrency exercises
CREATE DATABASE concurrency_lab;

\c concurrency_lab

-- Create tables for exercises
CREATE TABLE bank_accounts (
    account_id SERIAL PRIMARY KEY,
    customer_name VARCHAR(100),
    balance DECIMAL(10, 2) CHECK (balance >= 0),
    last_modified TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE inventory (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(100),
    quantity INTEGER CHECK (quantity >= 0),
    reserved INTEGER DEFAULT 0
);

CREATE TABLE audit_log (
    log_id SERIAL PRIMARY KEY,
    operation VARCHAR(50),
    table_name VARCHAR(50),
    record_id INTEGER,
    old_value TEXT,
    new_value TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert test data
INSERT INTO bank_accounts (customer_name, balance) VALUES
    ('John Doe', 1000.00),
    ('Jane Smith', 1500.00),
    ('Bob Johnson', 2000.00),
    ('Alice Williams', 750.00);

INSERT INTO inventory (product_name, quantity, reserved) VALUES
    ('Laptop', 10, 0),
    ('Mouse', 50, 0),
    ('Keyboard', 30, 0),
    ('Monitor', 15, 0);
```

Exercise 1: Simulate and Fix Lost Update

Objective: Experience a lost update problem and fix it.

Part A: Create the Problem

1. Open two terminal sessions (or psql connections)
2. In both terminals, connect to the database
3. Execute the following sequence:

Terminal 1:

```
BEGIN;
SELECT balance FROM bank_accounts WHERE customer_name = 'John Doe';
-- Note the balance
```

Terminal 2:

```
BEGIN;
SELECT balance FROM bank_accounts WHERE customer_name = 'John Doe';
-- Note the balance (should be same as Terminal 1)
```

Terminal 1:

```
-- John deposits $200
UPDATE bank_accounts
SET balance = balance + 200
WHERE customer_name = 'John Doe';
COMMIT;
```

Terminal 2:

```
-- John deposits $300 (from a different location)
UPDATE bank_accounts
SET balance = balance + 300
WHERE customer_name = 'John Doe';
COMMIT;

-- Check final balance
SELECT balance FROM bank_accounts WHERE customer_name = 'John Doe';
```

Questions:

1. What is the final balance?

2. What should the final balance be?
3. What happened and why?

Part B: Fix the Problem

Modify your approach using `SELECT ... FOR UPDATE` to ensure both deposits are applied correctly.

Write your solution here:

```
-- Terminal 1:
-- Your code here

-- Terminal 2:
-- Your code here
```

Exercise 2: Isolation Level Investigation

Objective: Understand how different isolation levels behave.

Part A: Read Committed (Default)

Terminal 1:

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT balance FROM bank_accounts WHERE customer_name = 'Jane Smith';
-- Record the value
```

Terminal 2:

```
BEGIN;
UPDATE bank_accounts SET balance = balance + 500 WHERE customer_name =
'Jane Smith';
COMMIT;
```

Terminal 1:

```
SELECT balance FROM bank_accounts WHERE customer_name = 'Jane Smith';
-- Record the value
COMMIT;
```

Questions:

1. Did the balance change between the two SELECT statements in Terminal 1?
2. Why or why not?

Part B: Repeatable Read

Repeat the same exercise but use `REPEATABLE READ` isolation level.

Terminal 1:

```
-- Your code here with REPEATABLE READ isolation
```

Questions:

1. Did the balance change between reads this time?
2. What's the difference between READ COMMITTED and REPEATABLE READ?
3. When would you use each isolation level?

Exercise 3: Deadlock Creation and Resolution

Objective: Create a deadlock situation and learn how to avoid it.

Part A: Create a Deadlock

Terminal 1:

```
BEGIN;
UPDATE bank_accounts SET balance = balance - 100 WHERE customer_name =
'Bob Johnson';
```

Terminal 2:

```
BEGIN;
UPDATE bank_accounts SET balance = balance - 50 WHERE customer_name =
'Alice Williams';
```

Terminal 1:

```
UPDATE bank_accounts SET balance = balance + 100 WHERE customer_name =
'Alice Williams';
-- This will wait...
```

Terminal 2:

```
UPDATE bank_accounts SET balance = balance + 50 WHERE customer_name = 'Bob
Johnson';
-- Deadlock should occur!
```

Questions:

1. What error message did you receive?
2. Which transaction was aborted?
3. Draw a diagram showing the circular wait condition.

Part B: Fix the Deadlock

Rewrite the transactions to avoid the deadlock by using a consistent locking order.

Your solution:

```
-- Terminal 1:
-- Your code here

-- Terminal 2:
-- Your code here
```

Exercise 4: Implement a Safe Money Transfer

Objective: Create a function that safely transfers money between accounts.

Write a SQL function that:

1. Transfers a specified amount from one account to another
2. Ensures sufficient funds exist before transferring
3. Prevents lost updates
4. Logs the transaction in the audit_log table
5. Handles errors appropriately

Your solution:

```
CREATE OR REPLACE FUNCTION transfer_money(
    from_customer VARCHAR(100),
    to_customer VARCHAR(100),
    amount DECIMAL(10, 2)
) RETURNS BOOLEAN AS $$
DECLARE
    -- Your variables here
BEGIN
    -- Your implementation here

    -- Return TRUE if successful, FALSE otherwise
END;
$$ LANGUAGE plpgsql;
```

Test your function:

```
-- Test 1: Valid transfer
SELECT transfer_money('Bob Johnson', 'Jane Smith', 100.00);
SELECT customer_name, balance FROM bank_accounts
WHERE customer_name IN ('Bob Johnson', 'Jane Smith');

-- Test 2: Insufficient funds
SELECT transfer_money('Alice Williams', 'John Doe', 10000.00);

-- Test 3: Concurrent transfers (open two terminals)
-- Terminal 1:
SELECT transfer_money('Bob Johnson', 'Alice Williams', 500.00);

-- Terminal 2 (simultaneously):
SELECT transfer_money('Bob Johnson', 'John Doe', 500.00);
```

Exercise 5: Inventory Management with Reservations

Objective: Implement a two-phase commit pattern for inventory management.

Create functions to:

1. Reserve inventory (mark items as reserved but not yet sold)
2. Confirm a sale (convert reservation to actual sale)
3. Cancel a reservation (return reserved items to available inventory)

Starter code:

```
CREATE OR REPLACE FUNCTION reserve_inventory(
    p_product_name VARCHAR(100),
    p_quantity INTEGER
) RETURNS INTEGER AS $$
DECLARE
    v_product_id INTEGER;
    v_available INTEGER;
BEGIN
    -- Your implementation here
    -- Return a reservation_id (could be a transaction id or generated
value)
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE FUNCTION confirm_sale(
    p_product_name VARCHAR(100),
    p_quantity INTEGER
) RETURNS BOOLEAN AS $$
BEGIN
    -- Your implementation here
    -- Actually subtract from quantity and clear reservation
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION cancel_reservation(
    p_product_name VARCHAR(100),
    p_quantity INTEGER
) RETURNS BOOLEAN AS $$
BEGIN
    -- Your implementation here
    -- Clear the reservation
END;
$$ LANGUAGE plpgsql;
```

Test scenario:

```
-- Simulate two customers trying to buy the last few laptops
-- Terminal 1:
BEGIN;
SELECT reserve_inventory('Laptop', 8);
-- Wait before confirming...

-- Terminal 2:
BEGIN;
SELECT reserve_inventory('Laptop', 8);
-- Should this succeed? How many are available?

-- Terminal 1:
SELECT confirm_sale('Laptop', 8);
COMMIT;

-- Terminal 2:
-- What should happen here?
COMMIT;
```

Exercise 6: Phantom Read Detection

Objective: Observe phantom reads and understand when they matter.

Part A: Demonstrate Phantom Reads

Terminal 1:

```
BEGIN TRANSACTION ISOLATION LEVEL READ COMMITTED;
SELECT COUNT(*) FROM bank_accounts WHERE balance > 1000;
SELECT SUM(balance) FROM bank_accounts WHERE balance > 1000;
-- Record both values
```

Terminal 2:

```
BEGIN;
INSERT INTO bank_accounts (customer_name, balance) VALUES ('New Customer',
1500.00);
COMMIT;
```

Terminal 1:

```
SELECT COUNT(*) FROM bank_accounts WHERE balance > 1000;
SELECT SUM(balance) FROM bank_accounts WHERE balance > 1000;
-- Record both values again
COMMIT;
```

Questions:

1. Did the count and sum change?
2. Repeat with REPEATABLE READ and SERIALIZABLE isolation levels. What happens?
3. In what business scenarios would phantom reads be a problem?

Exercise 7: Monitoring and Debugging Concurrency

Objective: Learn to diagnose concurrency issues.

1. In one terminal, start a long-running transaction that holds locks:

```
BEGIN;
UPDATE bank_accounts SET balance = balance + 1 WHERE customer_name = 'John
Doe';
SELECT pg_sleep(30); -- Simulate long operation
```

2. In another terminal, try to update the same row:

```
UPDATE bank_accounts SET balance = balance + 1 WHERE customer_name = 'John
Doe';
```

3. In a third terminal, run these diagnostic queries:

```
-- Query 1: Show all active locks
SELECT
    l.locktype,
    l.mode,
    l.granted,
    l.pid,
    a.usename,
    a.query,
```

```
    a.state
FROM pg_locks l
JOIN pg_stat_activity a ON l.pid = a.pid
WHERE a.datname = 'concurrency_lab';

-- Query 2: Show blocking locks
-- (Use the query from the examples file)

-- Query 3: Show lock wait times
SELECT
    pid,
    usename,
    wait_event_type,
    wait_event,
    state,
    query,
    NOW() - query_start AS wait_time
FROM pg_stat_activity
WHERE wait_event IS NOT NULL
    AND datname = 'concurrency_lab';
```

Questions:

1. Which locks are being held?
2. Which process is blocking which?
3. How would you resolve this situation in production?

Exercise 8: Optimistic Locking Implementation

Objective: Implement optimistic concurrency control.

1. Add version column to bank_accounts:

```
ALTER TABLE bank_accounts ADD COLUMN version INTEGER DEFAULT 1;
```

2. Create a function that uses optimistic locking:

```
CREATE OR REPLACE FUNCTION optimistic_update(
    p_customer_name VARCHAR(100),
    p_new_balance DECIMAL(10, 2),
    p_expected_version INTEGER
) RETURNS BOOLEAN AS $$
DECLARE
    v_rows_updated INTEGER;
BEGIN
    -- Your implementation here
    -- Return TRUE if update succeeded, FALSE if version conflict
END;
$$ LANGUAGE plpgsql;
```

3. Test with concurrent updates:

```
-- Terminal 1:
SELECT customer_name, balance, version FROM bank_accounts WHERE
customer_name = 'John Doe';
-- Note the version
SELECT optimistic_update('John Doe', 1100.00, 1);

-- Terminal 2 (simultaneously):
SELECT customer_name, balance, version FROM bank_accounts WHERE
customer_name = 'John Doe';
-- Note the version
SELECT optimistic_update('John Doe', 1200.00, 1);
-- Should one of these fail?
```

Questions:

1. What are the advantages of optimistic locking vs pessimistic locking?
2. When would you use each approach?

Bonus Challenge: Build a Ticket Booking System

Create a simple ticket booking system that:

- Tracks available seats for an event
- Allows multiple users to book seats concurrently
- Prevents overbooking
- Implements a timeout for reservations (reserved seats become available after 5 minutes if not confirmed)

Tables needed:

```
CREATE TABLE events (
    event_id SERIAL PRIMARY KEY,
    event_name VARCHAR(100),
    total_seats INTEGER,
    available_seats INTEGER
);

CREATE TABLE bookings (
    booking_id SERIAL PRIMARY KEY,
    event_id INTEGER REFERENCES events(event_id),
    customer_email VARCHAR(100),
    seats_reserved INTEGER,
    status VARCHAR(20), -- 'reserved', 'confirmed', 'cancelled'
    reserved_at TIMESTAMP,
    confirmed_at TIMESTAMP
);
```

Implement the following functions with proper concurrency control:

1. `reserve_seats(event_id, customer_email, num_seats)` - Reserve seats temporarily
2. `confirm_booking(booking_id)` - Confirm a reservation
3. `cancel_booking(booking_id)` - Cancel and release seats
4. `cleanup_expired_reservations()` - Release expired reservations

Test your implementation with multiple concurrent users trying to book seats for a popular event!

Submission Guidelines

For each exercise:

1. Provide your SQL code
2. Screenshot or copy the output
3. Answer all questions
4. Explain your reasoning for the solutions

Focus on understanding WHY things work the way they do, not just completing the exercises.